Supplementary Material

Ethan Lake

July 15, 2026

This file contains supplementary information supporting the main paper.

Contents

1 Size of the danger region

In this section we derive a simple formula for the size of the danger region quoted in the main text. Consider the erosion of a minority domain of size l under $\beta \to \infty$ sliding dynamics. The spacetime history of such a domain's erosion is shown in Fig. 1. One of the danger regions $\mathsf{DR}(l)$ is shown as the triangle **ABC**.

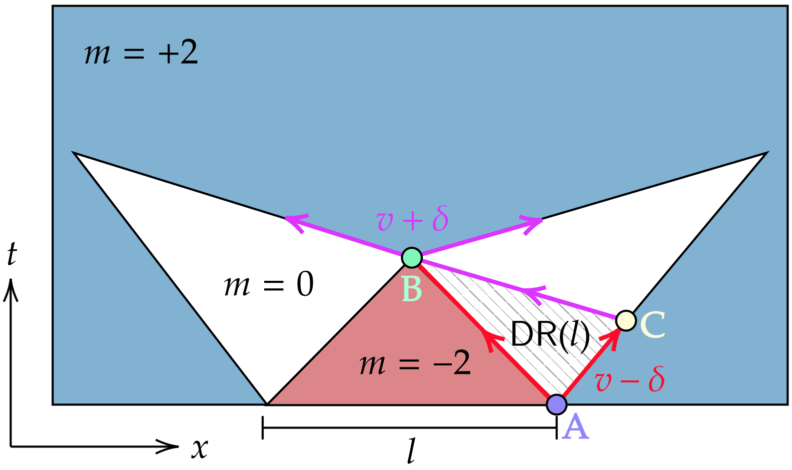


Figure 1: How a minority domain of size l is eroded under ideal sliding dynamics at $\beta \to \infty$, which maps to a version of Toom's two-line-voting automaton. The white regions have zero magnetization, and are metastable. The red arrows indicate interfaces that move with expected speed $v-\delta$, and the purple arrows those that move with expected speed $v+\delta$. The danger region $\mathsf{DR}(l)$ indicated by the triangle **ABC** is the spacetime region where spin flips in the metastable zero-magnetization area can propagate to prevent the erosion of the $m=-2$ (red) part of the initial minority domain.

Putting the origin of coordinates at the center of the minority domain, the spacetime coordinates of the points A, B in Fig. 1 are, by simple geometric considerations,

$$\begin{aligned}(t_A, x_A) &= (0, l/2)\\ (t_B, x_B) &= \left(\frac{l}{2(v-\delta)}, 0\right),\end{aligned} \tag{1}$$

and the coordinates of C are determined from the two equations

$$\begin{aligned}\frac{x_C - l/2}{v-\delta} &= t_C\\ \frac{x_C}{v+\delta} &= t_B - t_C,\end{aligned} \tag{2}$$

which gives

$$x_C - l/2 = \frac{l\delta}{2v}. \tag{3}$$

Therefore the area of the danger region is

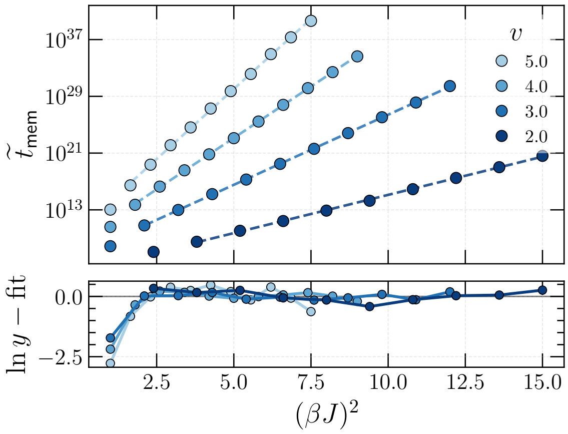
$$\begin{aligned}|\mathsf{DR}(l)| &= \frac{1}{2}|\mathbf{AC}\times\mathbf{AB}|\\ &= \frac{1}{2}|((x_C - l/2), t_C)\times(-l/2, t_B)|\\ &= \frac{l^2\delta}{4v(v-\delta)},\end{aligned} \tag{4}$$

giving the result quoted in the main text.

2 More data on the scaling of t_{mem}

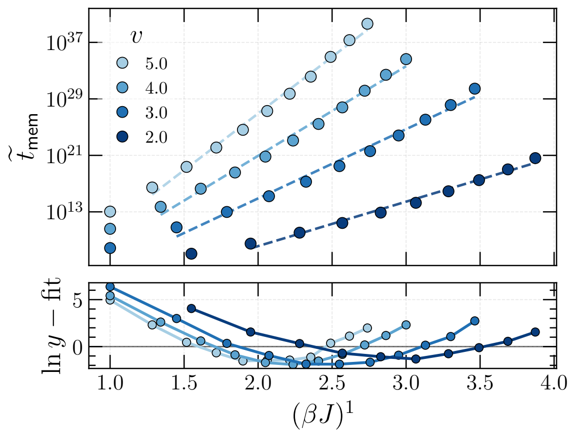
In this section, we provide more detail regarding what the numerics say about the scaling of the memory time with βJ and v.

First for the scaling with βJ. For the reasons explained in the main paper, we examine the nucleation-assisted memory time $\widetilde{t}_{\mathsf{mem}}$, with automatic injection of size 2 domains occurring in the maaximally magnetized states. Running a linear fit of $\ln \widetilde{t}_{\mathsf{mem}}$ to $(\beta J)^2$ gives



(5)

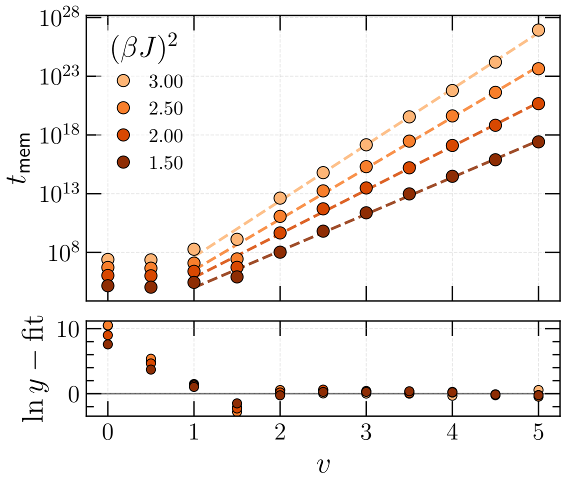
where the bottom panel shows the residuals of the fit. On the other hand, running a linear fit of $\ln \widetilde{t}_{\mathsf{mem}}$ to βJ gives



(6)

with clearly worse residuals and a systematic curvature away from a linear fit.

For the scaling with v, we re-plot the data in the main text, but now with the residual against a linear fit of $\ln \widetilde{t}_{\mathsf{mem}}$ to v. This gives



(7)

showing that the fit becomes good quality when $v > 2$, as expected.

3 Phase diagram in the $v \to \infty$ limit

When $v \to \infty$, the dynamics enters a simple mean-field limit: one chain couples randomly to sites of the other, seeing only (in expectation) the other chain's average magnetization (see also e.g. [1]). Since the two chains are statistically the same, we expect the dynamics in this limit to be governed by thermal dynamics with respect to the time-*independent* Hamiltonian

$$H = -\sum_i ((J\langle s\rangle + h)s_i + J s_i s_{i+1}) \tag{8}$$

where we have included an additional external magnetic field h for completeness. We then solve for $\langle s\rangle$ using a self-consistency equation derived from the exact expression for the average magnetization of a 1d Ising model in a field $\eta = J\langle s\rangle + h$. This is easily arrived at using the transfer matrix approach. Recall that the transfer matrix is

$$T = \begin{pmatrix} e^{\beta(J+\eta)} & e^{-\beta J} \\ e^{-\beta J} & e^{\beta(J-\eta)} \end{pmatrix}, \tag{9}$$

and the free energy density is

$$F = -\frac{1}{\beta}\ln\lambda_+, \tag{10}$$

with λ_+ the largest eigenvalue of T. Some algebra gives

$$F = -J - \frac{1}{\beta}\ln\left(\cosh(\beta\eta) + \sqrt{\cosh^2(\beta\eta) - 2e^{-2\beta J}\sin(2\beta J)}\right). \tag{11}$$

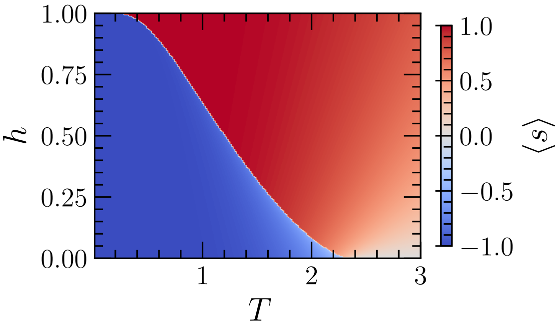
Taking the derivative with respect to η then gives the magnetization

$$m = \frac{\sinh(\beta\eta)}{\sqrt{\cosh^2(\beta\eta) - 2e^{-2\beta J}\sinh(2\beta J)}}. \tag{12}$$

In the present mean-field model, we thus have the self-consistent relation

$$\langle s\rangle = \frac{\sinh(\beta(J\langle s\rangle + h))}{\sqrt{\cosh^2(\beta(J\langle s\rangle + h) - 2e^{-2\beta J}\sinh(2\beta J)}}. \tag{13}$$

Working in units where $J = 1$ then gives the following phase diagram:



(14)

where we show the value of $\langle s\rangle$ in the less stable minimum (if multiple minima exist).

4 Details about forward-flux sampling

At low enough T and large enough v, our prediction for the scaling of t_{mem} rapidly becomes so large that confirming this prediction with a direct Monte-Carlo (MC) measurement of t_{mem} is infeasible. We instead estimate t_{mem} using forward-flux sampling (FFS) [2], a rare-event method that factorizes the long crossing time into a product of conditional crossing probabilities, each of which is $O(1)$ and can be measured directly. This section spells out how FFS is implemented in our code and how the various tuning parameters are chosen. All source files referenced below are in the public repository [3] (the FFS routines themselves live in `src/ffs.jl`).

4.1 Setup

We use the (signed) total magnetization per spin as the FFS order parameter (reaction coordinate),

$$m \equiv \frac{1}{2L}\sum_{i=1}^{L}\left(s_i^{(\mathsf{t})} + s_i^{(\mathsf{b})}\right) \in [-1, +1]. \tag{15}$$

Our simulations use single-spin Glauber dynamics at inverse temperature β, interleaved with the sliding shift: in our time units, one time step consists of $2L$ attempted spin flips interleaved with v rigid translations of the top chain by one lattice site, with the translations spaced at regular intervals (see `src/core.jl`).

The metastable basin A consists of configurations near the all-$+$ state, and the absorbing basin B consists of configurations with $m \le m_*$, where m_* is the `M_threshold` parameter in the code, which we take to be $1/2$ in numerics.

The intuition behind FFS is that even when the full barrier-crossing probability per unit time is very small in some control parameter, the *conditional* probability of advancing a small step up the barrier—given that the system has already made its way partway up—is of order unity. Concretely, we introduce a sequence of non-intersecting interfaces

$$m_0 > m_1 > m_2 > \cdots > m_n = m_* \tag{16}$$

in the m, lying between basin A and basin B. The mean first-passage time t_{mem} from A to B then admits the standard FFS factorization [2]

$$t_{\mathsf{mem}}^{-1} \approx \Phi_0 \prod_{k=1}^{n} P_k, \tag{17}$$

where Φ_0 is the flux of forward crossings of m_0 in trajectories confined to basin A, and P_k is the conditional probability that a trajectory which has just crossed m_{k-1} for the first time subsequently reaches m_k before returning to basin A. Each P_k and Φ_0 is by construction $O(1)$, so the right-hand side of (17) can be quickly evaluated even when t_{mem} itself is very large.

4.2 Phase 0: initial flux

The first ingredient is the basin-A flux Φ_0 through the first interface m_0. We initialize a configuration in the all-+ state and let it equilibrate within basin A for a short time (see `measure_initial_flux` in `src/ffs.jl`). We then run a long trajectory, monitoring $m(t)$ at every integer time step. Each time m crosses m_0 from above we record the configuration (this is a "forward crossing"); after such a crossing we wait for m to return above m_0 before counting the next one, so that consecutive crossings are independent excursions. If during one of these excursions m falls below the absorbing threshold m_* we terminate the trajectory and start a new one in basin A. Calling $\mathcal{T}$ the total time spent in basin A across all such trajectories and N_0 the number of forward crossings, we estimate

$$\Phi_0 \approx N_0/\mathcal{T}, \qquad \mathrm{Var}(\ln \Phi_0) \approx 1/N_0. \tag{18}$$

The ensemble $\mathcal{C}_0 = \{C_0^{(1)}, \ldots, C_0^{(N_0)}\}$ of recorded configurations serves as the source for the next phase. We run our simulations until the size of each $\mathcal{C}_k$ reaches a fixed value n_{configs}, set with the `--n_configs_per_run` command-line argument in the code. We use $n_{\text{configs}} \sim 15,000$ for the numerical results in the paper.

4.3 Placement of the first interface

The first interface m_0 should be far enough into basin A that crossings are reasonably frequent, but far enough from the all-+ state that the recorded configurations are genuinely partway up the barrier rather than typical fluctuations within A. We choose the interface magnetization adaptively by first measuring the metastable mean magnetization $\overline{m}_A$ and its standard deviation σ_A from a brief (ten time steps) run started in the all-+ state, and then setting

$$m_0 = \overline{m}_A - 2.5\,\sigma_A. \tag{19}$$

This puts m_0 in the lower tail of the metastable distribution: crossings occur often enough that Φ_0 converges quickly, but the configurations recorded already represent fluctuations distinguishable from typical A-states (if $m_0 \leq m_*$ then the transition is not actually a rare event and the FFS estimate is reported as failed).

4.4 Phase k: interface advancement

Given a source ensemble $\mathcal{C}_{k-1}$ at interface m_{k-1}, we measure P_k by firing trial trajectories from configurations drawn uniformly from $\mathcal{C}_{k-1}$. Each trial evolves under the same dynamics, with two stopping conditions:

- **success**: the trial reaches $m \leq m_k$, and its endpoint configuration is appended to the new source ensemble $\mathcal{C}_k$.
- **failure**: the trial returns above a *failure boundary* m_{fail}, defined below.

We continue firing trials until $|\mathcal{C}_k| = n_{\text{configs}}$ successful endpoint configurations have been collected. Letting N_k^{tot} be the total number of trials fired at phase k (successes plus failures),

$$P_k \approx n_{\text{configs}}/N_k^{\text{tot}}, \quad \text{Var}(\ln P_k) \approx (1 - P_k)/(P_k N_k^{\text{tot}}). \tag{20}$$

It occasionally proves numerically helpful to use a *lookback failure boundary* when defining m_{fail}: rather than declaring failure whenever a trial returns above m_0, we declare failure only when the trial returns above $m_{\max(k-K,0)}$, where $K = 7$ is a fixed integer (`n_lookback` in the code).

4.5 Adaptive interface placement

Equation (17) is exact for any choice of interfaces, but the variance of the estimator depends strongly on their placement. The optimal choice is to equalize the P_ks, since for fixed total computational budget the relative error is minimized when each interface costs the same number of trials. We achieve this by placing each new interface adaptively.

Given the source ensemble $\mathcal{C}_{k-1}$ at interface m_{k-1}, we fire n_{configs} exploratory trials. Each trial runs until it either falls below m_* or returns above the lookback failure boundary, and we record the minimum magnetization $m_{\min}^{(j)}$ reached by trial j. Sorting $\{m_{\min}^{(j)}\}$ in ascending order and letting $\alpha \in (0, 1)$ be a target conditional crossing probability (the `--target_crossing_prob` command-line argument; we use $\alpha = 0.25$ by default), we set

$$m_k = m_{\min}^{(\lceil \alpha n_{\text{configs}} \rceil)}. \tag{21}$$

By construction, a fraction $\approx \alpha$ of trajectories launched from $\mathcal{C}_{k-1}$ will reach m_k before returning, so $P_k \approx \alpha$ to within sampling noise.

As an alternative, the code also supports a *canonical* mode (the `--n_interfaces` command-line argument set to a positive integer) in which the interfaces are placed uniformly between m_0 and m_* and $m_{\text{fail}} = m_0$ is held throughout; this is useful in regimes where there is no genuine metastable barrier, but is not used for any data shown in the main text.

4.6 Combining the phases

Once all phases have run, the estimator (17) gives

$$\log t_{\mathsf{mem}} = -\log \Phi_0 - \sum_{k=1}^{n} \log P_k \tag{22}$$

with statistical variance

$$\mathrm{Var}(\ln t_{\mathsf{mem}}) \approx \frac{1}{N_0} + \sum_{k=1}^{n} \frac{1-P_k}{P_k N_k^{\mathrm{tot}}}. \tag{23}$$

The dominant cost of an FFS estimation is the number of interfaces n times the number of trials per interface. For the data shown in the main text we use $n_{\mathrm{configs}} = 15,000$ and usually have around 10-20 interfaces; with these choices, the data displayed in the main text can be obtained in a few tens of minutes on a laptop.

4.7 Isolating the nucleation rate

As discussed in the main text, relaxation in the systems we are concerned with is nucleation-limited rather than aggregation-limited. The logic is as follows: once a supercritical droplet forms at size ξ_{er}, it grows after a time t to size $\sim \xi_{\mathsf{er}} + \sqrt{Dt}$, coditioned on its size not dropping below ξ_{er} at any point; here D is the (noise-dependent) diffusion constant of the random walk executed by a well-isolated domain wall. The probability that the size of the droplet does not drop below ξ_{er} by time t is however $\sim 1/\sqrt{Dt}$, so the total size the droplet contributes in expectation is independent of D and t.

This means that the scaling of t_{mem} is set by the nucleation time of a supercritical droplet, and is independent of D to leading order. To estimate the scaling of t_{mem} most efficiently in FFS numerics, we therefore compute the memory time on a system of size $L = c\xi_{\mathsf{er}}$, where c is a constant of order unity (we fix $c = 5$ in our numerics).

4.8 Independent repetitions and error bars

To check that the variance estimate (23) is not dominated by tail effects or by an unlucky choice of source ensembles, we repeat the entire FFS procedure n_{rep} times for each choice of parameters point (this is the `--n_repeats` command-line argument; we use $n_{\mathrm{rep}} = 5$ for the data shown in the main text), each repeat using independent seeds and independent adaptive interface placements. We report the mean of $\log_{10} t_{\mathsf{mem}}$ across the n_{rep} runs, with error bars given by

$$\sigma_{\log t_{\mathsf{mem}}} = \max\left(\frac{1}{\ln 10}\sqrt{\frac{\overline{\mathrm{Var}(\ln t_{\mathsf{mem}})}}{n_{\mathrm{rep}}}}, \frac{\mathrm{std}(\log_{10} t_{\mathsf{mem}})}{\sqrt{n_{\mathrm{rep}}}} \right), \tag{24}$$

where the first argument is the (square root of the) mean intrinsic FFS variance (23) and the second is the empirical standard error across the n_{rep} repeats.

5 Memory in the GKL automaton

5.1 Setup and background

In this section we investigate the relaxation dynamics of the noisy GKL automaton [4] under noise of strength $\varepsilon \in [0,1]$ and bias $\eta \in [-1,1]$. In this dynamics, the spins all update synchronously in discrete time, and each spin is subjected to an error with probability ε. When an error occurs, a spin is replaced by ± 1 with probabiblity $(1 \pm \eta)/2$.

It was proven in [5] that there is a positive constant $\eta_c > 0$ such that

$$t_{\mathsf{mem}} = e^{\Theta(\log^2 \varepsilon)} \qquad \forall\, \eta \in [-1, -\eta_c] \cup [\eta_c, 1], \tag{25}$$

where Θ denotes scaling with respect to ε. The general proof strategy is quite simple to summarize: following a similar argument as in Sec. 1, one concludes that the erosion length in the GKL automaton is, in the $\varepsilon \ll 1$ limit

$$\xi_{\mathsf{er}} \sim \frac{1}{\sqrt{\varepsilon}}, \tag{26}$$

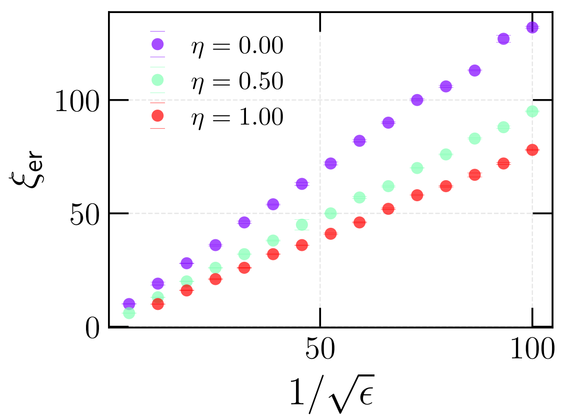
where the $\sim$ omits ε-independent factors. Since an error that occurs in the danger region can grow a minority domain of size l to one of size λl with $\lambda > 1$, growing a domain to size ξ_{er} only requires $\log_\lambda \xi_{\mathsf{er}}$ steps. Thus

$$t_{\mathsf{mem}} \sim \varepsilon^{-\log_\lambda \xi_{\mathsf{er}}} = e^{-\Theta(\log^2 \varepsilon)}. \tag{27}$$

While Ref. [5] did not make rigorous statements about the case of unbiased noise, the physical arguments above do not depend crucially on taking $\eta \neq 0$. However, in [6] it was proposed from direct Monte Carlo numerics that in fact $t_{\mathsf{mem}} = e^{\Theta(1/\varepsilon)}$ when $\eta = 0$. Unfortunately, distinguishing between $e^{\Theta(1/\varepsilon)}$ and $e^{\Theta(\log^2 \varepsilon)}$ is very difficult in naive Monte Carlo, especially with the limited range of ε values accessed in [6]. In what follows, we revisit this question with more precise numerics.

5.2 Scaling of the erosion length

First, we numerically confirm that (26) holds regardless of η. This is done using the same procedure as used for the sliding Ising model, and gives

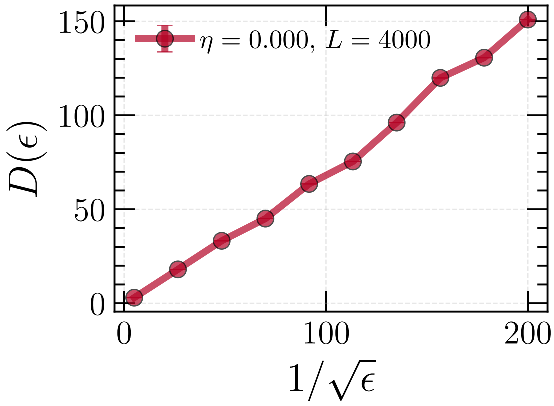


(28)

as predicted.

5.3 Domain wall diffusion

Once formed, a supercritical droplet of size $l > \xi_{\mathsf{er}}$ has endpoints which undergo biased diffusion, with the sign of the bias determined by η. We can numerically estimate $D(\varepsilon)$ by preparing a system in a state with $s_{i<L/2} = +1, s_{i\geq L/2} = -1$, and computing the diffusion constant of the domain wall position $x_{dw} = \sum_i (s_i+1)/2$. Doing this at zero bias and with a system of size $L = 4000$ gives $D(\varepsilon) \propto 1/\sqrt{\varepsilon}$:

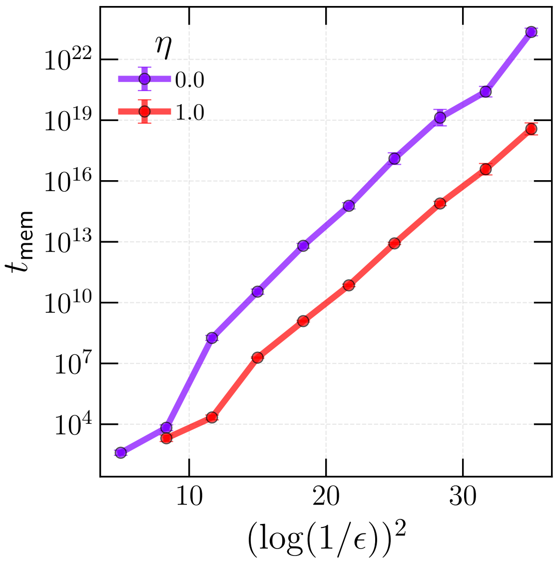


. (29)

The behavior of D is tangential to the discussion of t_{mem} however, since relaxation is nucleation-limited rather than coalesence-limited, for the same reason as in the sliding Ising chain context, c.f. Sec. 4.7.

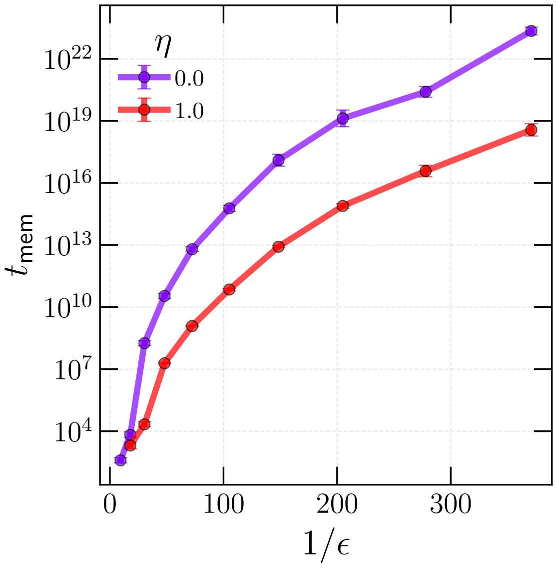
5.4 Scaling of t_{mem}

We now numerically check (27) with the FFS technique, using the adaptive system size protocol of Sec. 4.7. We set $m_* = 1/2$ and $n_{\text{configs}} = 15,000$. Plotting t_{mem} against $(\log(1/\varepsilon))^2$, we find



(30)

The same data plotted against $1/\varepsilon$ clearly reveal that (27) fits the data better:



(31)

References

[1] D. Kadau, A. Hucht, and D. E. Wolf, Physical review letters **101**, 137205 (2008).

[2] R. J. Allen, C. Valeriani, and P. Rein ten Wolde, Journal of physics: Condensed matter **21**, 463102 (2009).

[3] See the following github repository for the supplementary material and full codebase: `github.com/ethanlake/sliding-layers` (2026).

[4] P. Gacs, G. L. Kurdyumov, and L. A. Levin, Problemy Peredachi Informatsii **14**, 92 (1978).

[5] K. Park, *Ergodicity and mixing rate of one-dimensional cellular automata*, Tech. Rep. (Boston University Computer Science Department, 1997).

[6] P. G. de Sá and C. Maes, Journal of Statistical Physics **67**, 507 (1992).